

Tensores y álgebra computacional con PyTorch

Correspondencia entre ejes, índices, forma algebraica e implementación computacional.

EJES

qué representa cada dimensión

ÍNDICES

qué se conserva o se contrae

CÓDIGO

qué relación se ejecuta

Las formas no bastan: cada eje necesita significado

Antes de operar, registre qué cantidad representa cada eje.

REGISTRO SEMÁNTICO DEL CASO CONDUCTOR

SÍMBOLO	FORMA	EJE 0	EJE 1	ÍNDICES
X	(B, D)	observación	característica de entrada	i, d
W	(D, H)	característica de entrada	característica de salida	d, h
b	$(H,)$	característica de salida	—	h
Y	(B, H)	observación	característica de salida	i, h

✓ Una forma como (B, D) no identifica observaciones ni características.

El índice d desaparece; i y h determinan la salida

La forma se obtiene identificando qué índices permanecen libres.

ECUACIÓN POR COMPONENTES

$$Y_{ih} = \sum_{d=1}^D X_{id} W_{dh} + b_h$$

SOBREVIVE

i

ÍNDICE LIBRE
conserva observaciones
reaparece como eje de salida

SE SUMA

d

ÍNDICE CONTRAÍDO
se suma y desaparece
no crea un eje en el resultado


SOBREVIVE

h

ÍNDICE LIBRE
conserva salidas
reaparece como eje de salida

FORMA PREDICHA

$$Y \in \mathbb{R}^{B \times H}$$

Marca el índice que se suma y los que deben reaparecer en el resultado.

Un tensor matemático no es su almacenamiento

Las componentes aparecen después de elegir bases.



El objeto T no es un arreglo particular $[T]_{\mathcal{B}}$.

Un mismo tensor puede tener componentes distintas

El objeto permanece fijo; sus componentes numéricas dependen de la base elegida.



$[T]_{\mathcal{B}}$ y $[T]_{\mathcal{B}'}$ pueden diferir
Mismo T ; la representación depende de la base elegida.

shape y dtype describen la representación, no la semántica

Los metadatos verifican estructura; el significado de los ejes sigue siendo un contrato externo.

TRADUCCIÓN ENTRE MATEMÁTICA Y REPRESENTACIÓN

$T_{i_1 \dots i_k} \leftrightarrow$ valor indexado	$(n_1, \dots, n_k) \leftrightarrow$ shape	$k \leftrightarrow$ ndim	dominio numérico $\leftrightarrow$ dtype aproximado
--	---	--------------------------	---

COMPROBACIÓN POSTERIOR A LA PREDICCIÓN



shape, dtype, device y layout describen estructura; la semántica de ejes es externa.

Orden, forma y tamaño de eje responden preguntas distintas

Cada descriptor responde una pregunta diferente sobre el mismo tensor.

MISMO OBJETO · CUATRO PREGUNTAS

$$T \in \mathbb{R}^{n_1 \times \dots \times n_k}$$

La notación concentra estructura; las tarjetas separan sus significados.

1 ORDEN

¿Cuántos ejes tiene?

 k

un número

2 FORMA

¿Cuánto mide cada eje?

 $(n_1, \dots, n_k)$

una tupla ordenada

3 EJE r

¿Cuánto mide este eje?

 n_r

un tamaño particular

4 COMPONENTE

¿Qué valor selecciono?

 $T_{i_1 \dots i_k}$

todos los índices fijos



EVITE DOS ATAJSOS AMBIGUOS

No use "rank" para contar ejes ni confunda la forma completa con el tamaño de un eje.

Fijar un índice elimina un eje; cortar puede conservarlo

La forma se deriva identificando qué índices siguen libres.

TENSOR DE PARTIDA

$$S \in \mathbb{R}^{B \times T \times D}$$

i = observación

t = tiempo

d = característica

FIJAR i

se elimina observación

$$S_{i::} \quad S[0]$$

$$(T, D) = (5, 2)$$

quedan libres t, d

FIJAR t

se elimina tiempo

$$S_{:t:} \quad S[:, 0, :]$$

$$(B, D) = (4, 2)$$

quedan libres i, d

FIJAR d

se elimina característica

$$S_{::d} \quad S[:, :, 0]$$

$$(B, T) = (4, 5)$$

quedan libres i, t



INDEXAR NO ES LO MISMO QUE CORTAR

Un índice escalar elimina el eje; un intervalo puede conservarlo aunque seleccione un solo elemento.

Prediga tres selecciones antes de inspeccionar `.shape`

La aserción es evidencia posterior: primero derive la forma desde los índices.

REGLA DE TRABAJO

selección → índices libres → forma predicha → comprobación

1 FIJA *i*

$S[0] \rightarrow (5, 2)$

libres: *t*, *d*

```
assert S[0].shape == (5, 2)
```

2 FIJA *t*

$S[:, 0, :] \rightarrow (4, 2)$

libres: *i*, *d*

```
assert S[:, 0, :].shape == (4, 2)
```

3 FIJA *d*

$S[:, :, 0] \rightarrow (4, 5)$

libres: *i*, *t*

```
assert S[:, :, 0].shape == (4, 5)
```

UNA ASERCIÓN EXITOSA NO SUSTITUYE LA EXPLICACIÓN

Indique qué eje desapareció y por qué los demás permanecen.

Hadamard conserva índices; el producto exterior crea dos

La misma clase de salida no implica el mismo mecanismo algebraico.

1 PRODUCTO DE HADAMARD

multiplica componente a componente

$$C_{ij} = A_{ij}B_{ij}$$

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 0 \\ -1 & 5 \end{bmatrix}$$

$$A \odot B = \begin{bmatrix} 2 & 0 \\ -3 & 20 \end{bmatrix}$$

sin suma · conserva i, j · forma (m,n)

2 PRODUCTO EXTERIOR

combina cada componente de u con cada componente de v

$$C_{ij} = u_i v_j$$

$$u = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad v = \begin{bmatrix} 3 & -1 & 4 \end{bmatrix}$$

$$uv^T = \begin{bmatrix} 3 & -1 & 4 \\ 6 & -2 & 8 \end{bmatrix}$$

crea i, j libres · forma (m,n)



MIRE EL PATRÓN DE ÍNDICES ANTES DE MIRAR SOLO LA FORMA

Hadamard empareja posiciones existentes; el exterior forma todas las parejas posibles.

Permutar reordena; reducir elimina un índice

Observe qué sucede con los índices antes de mirar la función.

MISMO TENSOR DE PARTIDA

$$S \in \mathbb{R}^{B \times T \times D}$$

Los índices i, t, d tienen una función distinta en cada ruta.

P

PERMUTAR

Conserva los valores; cambia el orden de ejes.

$$P_{dit} = S_{itd} \quad P : (D, B, T)$$

`S.permute(2,0,1)`

t sigue presente

Σ

REDUCIR

Agrega componentes; elimina el índice sumado.

$$R_{id} = \sum_{t=1}^T S_{itd} \quad R : (B, D)$$

`S.sum(dim=1)`

t desaparece

Reordenar no agrega; reducir no es una permutación.

Cuatro operaciones, cuatro efectos sobre los índices

Complete efecto y forma antes de contrastar la ejecución.

OPERACIÓN	PATRÓN DE ÍNDICES	EFFECTO	FORMA
1 HADAMARD	$C_{ij} = A_{ij}B_{ij}$	efecto / índices: _____	$(_, _, _)$
2 EXTERIOR	$C_{ij} = u_i v_j$	efecto / índices: _____	$(_, _, _)$
3 PERMUTE	$P_{dit} = S_{itd}$	efecto / índices: _____	$(_, _, _, _)$
4 SUM	$R_{id} = \sum_t S_{itd}$	efecto / índices: _____	$(_, _, _)$

EJECUTE SOLO DESPUÉS DE COMPLETAR LA TABLA

A*B → shape ?

outer(u,v) → shape ?

permute → shape ?

sum → shape ?

Broadcasting compara formas desde el último eje

La comparación comienza en el extremo derecho, no en el primero.

DIRECCIÓN DE COMPARACIÓN				
				
				último eje
FORMA A	5	1	4	1
FORMA B	EJE AUSENTE	3	1	1
REGLA	se completa con 1	uno se expande	uno se expande	tamaños iguales
RESULTADO	5	3	4	1

Compatible por eje: iguales, uno de ellos es 1 o el eje no existe.

$XW + b$ es válido porque b_h actúa sobre la salida h

La compatibilidad dimensional es válida porque coincide con el significado de los ejes.

CONDICIÓN SEMÁNTICA

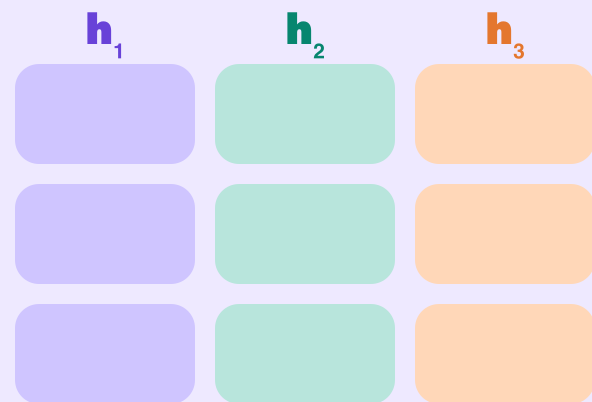
El mismo desplazamiento de salida se aplica a todas las observaciones.

SALIDA ANTES DEL SESGO

filas: observaciones i

columnas: salidas h

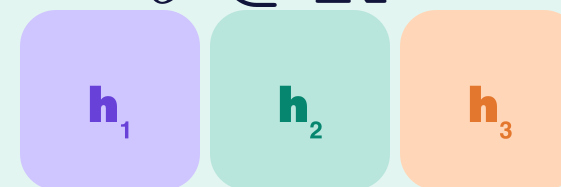
$$XW \in \mathbb{R}^{B \times H}$$



VECTOR DE SESGO

un valor por salida h

$$b \in \mathbb{R}^H$$



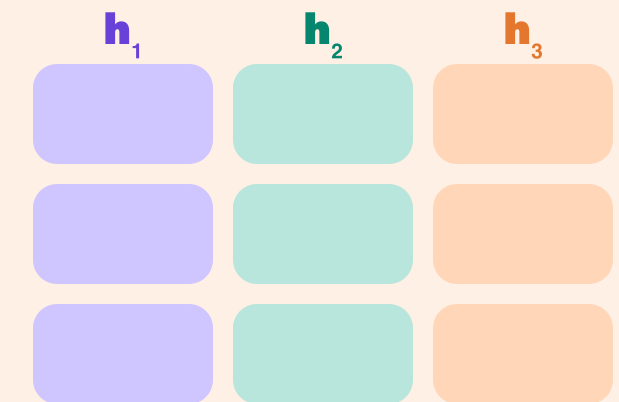
se replica sobre i



EXPANSIÓN EXPLÍCITA

misma columna h en cada fila i

$$\mathbf{1}_B b^\top \in \mathbb{R}^{B \times H}$$



MISMA RELACIÓN, DOS ESCRITURAS

$$(XW + b)_{ih} = (XW)_{ih} + b_h$$



$$XW + b = XW + \mathbf{1}_B b^\top$$

$B = H$ puede ocultar un ajuste por el eje equivocado

Que una operación ejecute no demuestra que alinee la cantidad correcta.

INTENCIÓN DECLARADA

Un ajuste diferente para cada observación
el índice que debe variar es i

$$Q_{ih} = Z_{ih} + r_i$$

DATOS COMPARTIDOS • ACCIDENTE

$$Z = 0_{3 \times 3}, \quad r = [10 \ 20 \ 30]^T$$

$$B = H = 3$$

×

EJECUTA, PERO CAMBIA EL EJE

$$\text{wrong} = Z + r$$

$$(Z + r)_{ih} = Z_{ih} + r_h$$

PRIMERA FILA

$$[10 \ 20 \ 30]$$

r se replica por filas y varía con h

✓

CORRIGE LA REPRESENTACIÓN

$$\text{right} = Z + r[:, \text{None}]$$

$$Q_{ih} = Z_{ih} + r_i$$

PRIMERA FILA

$$[10 \ 10 \ 10]$$

r se replica por columnas y varía con i

Compatibilidad dimensional $\neq$ validez semántica: identifique el índice antes de confiar en la ejecución.

PyTorch confirma XW antes de incorporar el sesgo

La matriz esperada se calcula primero; el framework solo la contrasta.

ORDEN DE VALIDACIÓN

1 predecir valores

2 ejecutar

3 explicar la discrepancia

OBSERVACIONES × RASGOS

X tiene forma (3,2)

$$X = \begin{bmatrix} 1 & 2 \\ 0 & -1 \\ 3 & 1 \end{bmatrix}$$

×

RASGOS × SALIDAS

W tiene forma (2,3)

$$W = \begin{bmatrix} 2 & -1 & 0 \\ 1 & 1 & 2 \end{bmatrix}$$

=

MATRIZ ESPERADA

XW tiene forma (3,3)

$$\text{expected}_{XW} = \begin{bmatrix} 4 & 1 & 4 \\ -1 & -1 & -2 \\ 7 & -2 & 2 \end{bmatrix}$$

CONTRACCIÓN VISIBLE

$$(XW)_{ih} = \sum_{d=1}^D X_{id}W_{dh}$$

d desaparece; i,h permanecen

CONTRASTE EN PYTORCH

$XW = X @ W$

`assert torch.equal(XW, expected_XW)`

Una aserción exitosa confirma la predicción; no la reemplaza.

TensorFlow ofrece el mismo contraste conceptual, no otra ruta

La formulación es común; la sintaxis cambia sin crear una segunda ruta del curso.

CONTRATO COMPARTIDO

$$X : (B, D), \quad W : (D, H), \quad b : (H,), \quad Y : (B, H)$$

observaciones i · rasgos d · salidas h

PYTORCH

implementación principal de M07

$$Y = X @ W + b$$

Se usa para construir y verificar.

La forma y los valores se contrastan con aserciones.

MISMA
RELACIÓN



TENSORFLOW

traducción conceptual única

$$Y_tf = tf.linalg.matmul(X_tf, W_tf) + b_tf$$

Se usa para reconocer la misma idea.

No abre un catálogo de API ni compara rendimiento.

INVARIANTE CONCEPTUAL

$$Y_{ih} = \sum_{d=1}^D X_{id} W_{dh} + b_h$$

La formulación decide; el framework contrasta.

Hadamard y matmul responden preguntas distintas

Las mismas entradas pueden definir mecanismos y salidas diferentes.

ENTRADAS COMPARTIDAS

A tiene filas i y columnas j ; u comparte el índice j . $A \in \mathbb{R}^{2 \times 3}$, $u \in \mathbb{R}^3$



HADAMARD

multiplica componente a componente

$$C_{ij} = A_{ij}u_j$$

i, j libres

i, j libres

$A * u$

salida (2,3)

No aparece una suma: j permanece.



MATMUL

combina columnas de A con entradas de u

$$y_i = \sum_{j=1}^3 A_{ij}u_j$$

i libre • j sumado

solo i

$A @ u$

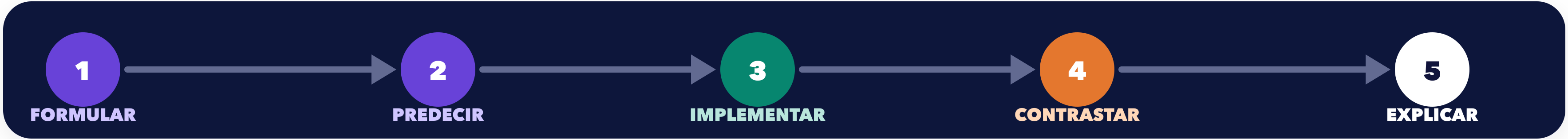
salida (2,)

La suma elimina j del resultado.

La forma sigue al patrón de índices: conservar j no equivale a contraerlo.

La síntesis exige predecir forma y componentes antes de ejecutar

Complete la predicción; use el código únicamente como contraste posterior.



TICKET DE SALIDA • COMPLETE ANTES DE EJECUTAR

OPERACIÓN		COMPONENTES	FORMA
A * u	1		
torch.outer(u, v)	2		
A @ u	3		
T.sum(dim=0)	4		
T.permute(2,0,1)	5		

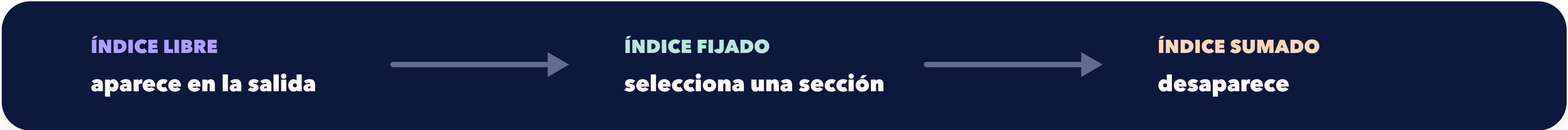
PUERTA DE EJECUCIÓN

El código se habilita cuando las cinco filas tienen componentes y forma.

RUN **bloqueado**

Cada índice libre sobrevive; cada índice sumado desaparece

Recupere la regla desde la notación; no consulte la sesión anterior.

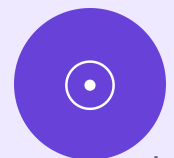


CLASIFIQUE CADA EXPRESIÓN	LIBRES	SUMADOS	FORMA
$P_{ij} = A_{ij}B_{ij}$	_____	_____	_____
$Q_{ij} = u_iv_j$	_____	_____	_____
$r_i = \sum_j A_{ij}$	_____	_____	_____
$C_{ik} = \sum_j A_{ij}B_{jk}$	_____	_____	_____

La forma se reconstruye desde los índices; no se adivina desde la sintaxis.

La forma final no identifica por sí sola la operación

Dos matrices pueden compartir dimensiones sin compartir la regla que las construye.



HADAMARD

combina entradas con los mismos índices

$$A, B \in \mathbb{R}^{m \times n}$$

$$P_{ij} = A_{ij}B_{ij}$$



PRODUCTO EXTERIOR

crea todas las parejas de índices i, j

$$u \in \mathbb{R}^m, \quad v \in \mathbb{R}^n$$

$$Q_{ij} = u_i v_j$$

MISMA FORMA



(m, n)

QUÉ DEBE DISTINGUIRSE

Hadamard conserva índices existentes; el exterior crea el par libre i, j .

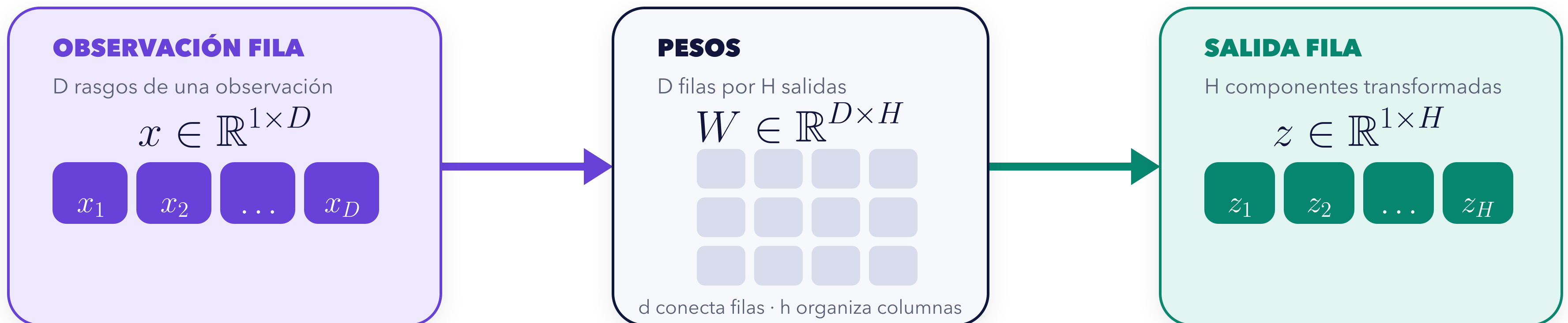
La forma coincide; el mecanismo matemático no.

Una observación fila se transforma como xW

Fijamos una sola convención: cada observación ocupa una fila.

CONVENCIÓN DE M07

observación = fila · rasgos d · salidas h



CONTRACCIÓN POR COMPONENTES

$$(1, D) @ (D, H) \longrightarrow (1, H) \qquad z_h = \sum_{d=1}^D x_d W_{dh}$$

d aparece en ambos factores y desaparece; h permanece en la salida.

Agregar b convierte la aplicación en afín

La prueba en el origen separa las dos clases sin depender de la implementación.

PRUEBA NECESARIA DE LINEALIDAD

Una aplicación lineal debe enviar el vector cero al vector cero.

$\mathbf{x} = \mathbf{0}$

L APLICACIÓN LINEAL

$$L(x) = xW \implies L(0) = 0$$

f APLICACIÓN AFÍN

$$f(x) = xW + b^T \implies f(0) = b^T \neq 0$$

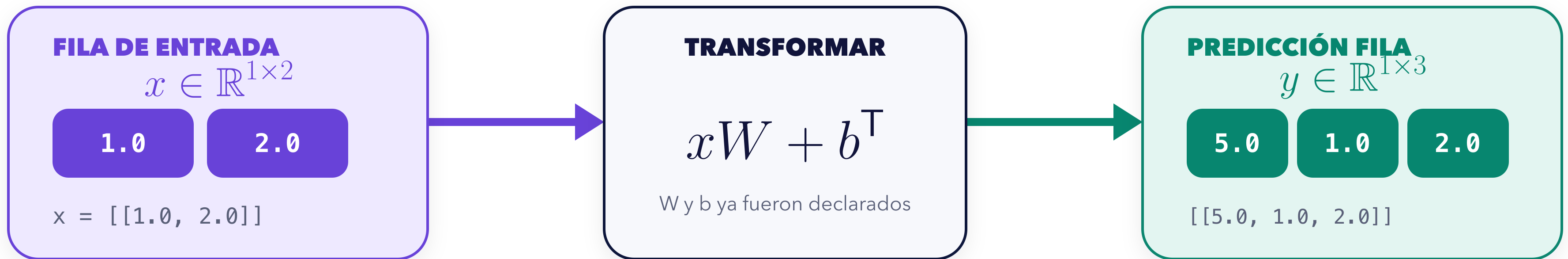
Si el desplazamiento es no nulo, la aplicación no preserva el origen: es afín, no lineal.

La comprobación conserva $(1, D) \rightarrow (1, H)$

Primero se predicen forma y valores; después se permite ejecutar.

CONTRATO DE FORMA

La observación sigue siendo una fila bidimensional durante toda la comprobación.



EJECUTAR Y EXIGIR DOS EVIDENCIAS

```
assert tuple(observed.shape) == (1, 3)
```

```
assert torch.equal(observed, prediction)
```

Preservar la fila es parte del significado: $(1, 3)$ no es $(3,)$

El lote aplica la misma transformación a cada fila

El índice de observación cambia; los parámetros de la transformación no.

CONTRATO DEL LOTE

Cada fila recibe el mismo W y el mismo b .



REGLA POR COMPONENTES

$$Y_{ih} = \sum_{d=1}^D X_{id} W_{dh} + b_h$$

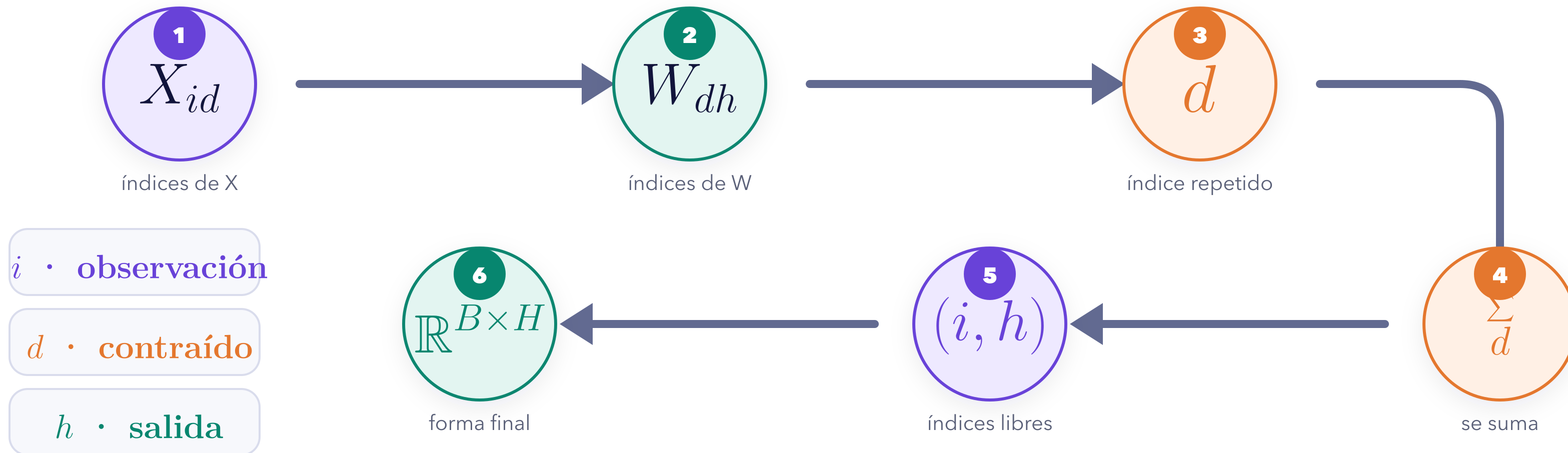
El índice i identifica la observación y permanece en la salida; W y b no dependen de i .

d se contrae; i y h sobreviven

La forma de salida se obtiene siguiendo índices, no probando operaciones en la API.

PUNTO DE PARTIDA

El índice d aparece en ambos factores y se suma.



RESULTADO DE LA DERIVACIÓN

$$Y_{ih} = \sum_{d=1}^D X_{id} W_{dh} + b_h$$

b_h se replica sobre i sin cambiar la forma.

$\mathbf{1}_B b^\top$ hace explícito lo que broadcasting abrevia

La forma explícita revela qué eje se replica y cuál conserva su significado.

IDENTIDAD POR COMPONENTES

$$(\mathbf{1}_B b^\top)_{ih} = (\mathbf{1}_B)_i b_h = b_h$$

REPLICADOR DE FILAS

$$\mathbf{1}_3 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

una entrada por observación

×

DESPLAZAMIENTO DE SALIDA

$$b^\top = [1 \ 0 \ -2]$$

la misma fila para cada observación



MATRIZ EXPLÍCITA DEL SESGO

$$\mathbf{1}_3 b^\top = \begin{bmatrix} 1 & 0 & -2 \\ 1 & 0 & -2 \\ 1 & 0 & -2 \end{bmatrix}$$

filas: observación i columnas: salida h

LECTURA SEMÁNTICA

El índice i selecciona la fila; h conserva la salida. Cada fila recibe el mismo b_h .

La forma explícita y la abreviada producen el mismo Y

La igualdad numérica depende de interpretar b como desplazamiento de la salida h .

RUTA EXPLÍCITA

la expansión del sesgo permanece visible

$$XW = \begin{bmatrix} 4 & 1 & 4 \\ -1 & -1 & -2 \\ 7 & -2 & 2 \end{bmatrix} + \mathbf{1}_3 b^\top = \begin{bmatrix} 1 & 0 & -2 \\ 1 & 0 & -2 \\ 1 & 0 & -2 \end{bmatrix}$$

RUTA BROADCAST

$$XW + b, \quad b : (3,)$$

$b : (H,)$ se alinea con h y se replica sobre i

RESULTADO COMÚN

$$Y = \begin{bmatrix} 5 & 1 & 2 \\ 0 & -1 & -4 \\ 8 & -2 & 0 \end{bmatrix}$$

índices i, h · forma $(3, 3)$

DOS COMPROBACIONES, DESPUÉS DEL CÁLCULO

```
assert torch.equal(explicit, broadcasted)
```

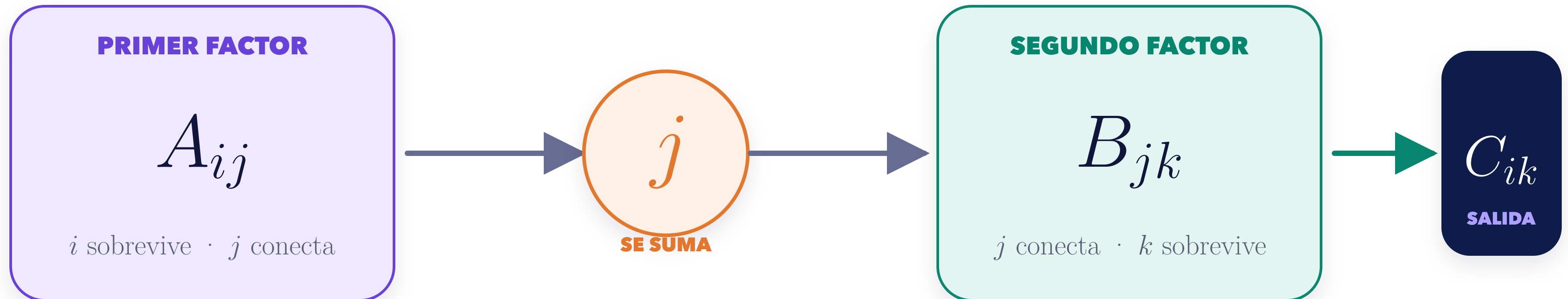
```
assert torch.equal(broadcasted, expected_Y)
```

El producto matricial contrae el eje compartido

El índice repetido conecta ambos factores, se suma y desaparece del resultado.

REGLA POR COMPONENTES

$$C_{ik} = \sum_{j=1}^n A_{ij} B_{jk}$$



$$A : (m, n) \quad @ \quad B : (n, p) \quad \longrightarrow \quad C : (m, p)$$

La compatibilidad de tamaños en j no garantiza compatibilidad semántica.

El tiempo se conserva cuando no participa en la contracción

Los ejes de contexto atraviesan la operación; solo d entra en la suma.

CONTRACCIÓN CON DOS EJES DE CONTEXTO

$$U_{ith} = \sum_{d=1}^D S_{itd} W_{dh}$$



i, t permanecen libres; d se elimina; h aparece como salida.

matmul usa los dos últimos ejes como matrices

Los ejes anteriores forman el lote; solo el eje compartido participa en la suma.

REGLA MATEMÁTICA

$$U_{ith} = \sum_{d=1}^D S_{itd} W_{dh}$$

TENSOR CON EJES DE LOTE

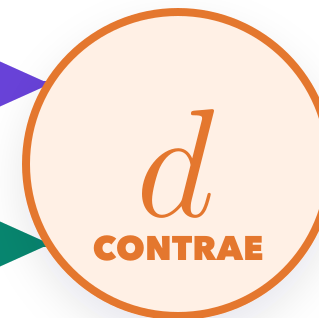
$$S : (B, T \mid D) = (4, 5 \mid 2)$$

B y T quedan fuera de la multiplicación matricial.

MATRIZ COMPARTIDA

$$W : (D \mid H) = (2 \mid 3)$$

La misma W se aplica a cada par (i,t).



FORMA PREDICHA

$$U : (B, T \mid H) = (4, 5 \mid 3)$$

```
assert tuple(U.shape) == (4, 5, 3)
```

PREDECIR → EJECUTAR → VERIFICAR

```
predicted_shape = (B, T, H) | U = torch.matmul(S, W)
```

dtype y device describen ejecución, no significado

Una conversión explícita cambia cómo se calculan los valores, no qué representa cada eje.



LÍMITE SEMÁNTICO

Ni dtype ni device dicen cuál eje contiene observaciones o características.

```
assert T.dtype == torch.int64 assert T_float.dtype == torch.float32 assert T.device.type == "cpu"
```

reshape puede copiar; view exige compatibilidad de almacenamiento

La forma resultante puede ser predecible aunque el alias y el almacenamiento no lo sean.

PUNTO DE PARTIDA

base : (3, 4) $3 \cdot 4 = 12$ elementos

RUTA A • RESHAPE

flat:(12,) = base.reshape(12)

¿VISTA O COPIA?

No está garantizado.

RUTA B • PERMUTE + VIEW

permuted:(4,3) = base.permute(1, 0)

VIEW EXIGE COMPATIBILIDAD

Tamaños y strides deben permitirla.

PROTOCOLO DE VERIFICACIÓN

SHAPE

¿es la esperada?

SEMÁNTICA

¿qué significa?

N.º ELEMENTOS

¿se preserva?

EJECUCIÓN

¿vista o copia?

Código válido puede sumar el vector sobre el eje incorrecto

El nombre de una variable no controla la regla de alineación que ejecuta la API.

MISMA FORMA EXTERNA

```
scores.shape = (3, 3) offset.shape = (3,) candidate.shape = (3, 3)
```

INTENCIÓN DECLARADA

Un desplazamiento diferente para cada observación.

$$Q_{ih} = Z_{ih} + r_i$$

EL ÍNDICE DEL VECTOR DEBE SER i

CONDICIÓN

$$B = H = 3$$

oculta el error

OPERACIÓN EJECUTADA

La API alinea el vector con el último eje.

`candidate = scores + offset`

$$\text{candidate}_{ih} = Z_{ih} + r_h$$

LA API USA h

DIAGNÓSTICO

Misma shape + ejecución exitosa ≠ corrección semántica

Hay que declarar qué eje modifica el vector antes de aceptar el resultado.

Una aserción semántica distingue el candidato de la corrección

La prueba debe detectar un cambio de significado, no solo una excepción.

CONTRATO SEMÁNTICO

`corrected = scores + offset_by_observation[:, None]`

$$\text{corrected}_{ih} = \text{scores}_{ih} + \text{offset}_i$$

PREDICCIÓN EXPLÍCITA

La matriz esperada fija valores y orientación.

$$\text{expected} = \begin{bmatrix} 11 & 12 & 13 \\ 24 & 25 & 26 \\ 37 & 38 & 39 \end{bmatrix}$$

`shape (3, 3)`

DOS ASERCIONES COMPLEMENTARIAS



`assert torch.equal(corrected, expected)`

Confirma la relación declarada.



`assert not torch.equal(candidate, corrected)`

Detecta la suma sobre el eje equivocado.

CRITERIO DE REGRESIÓN

La prueba debe fallar si cambia el significado, aunque el código siga ejecutando.

EJECUTAR ≠ VALIDAR

La cadena completa une índices, matriz, forma y código

Una implementación es trazable cuando conserva cantidades, ejes y relación.

1 • CANTIDADES Y EJES

$X : (B, D) = (3, 2)$ $W : (D, H) = (2, 3)$ $b : (H,) = (3,)$ i : observación d : entrada h : salida

2 • COMPONENTES

La suma identifica el eje compartido.

$$Y_{ih} = \sum_{d=1}^D X_{id} W_{dh} + b_h$$

d SE CONTRAE

i, h SON LIBRES

Observaciones por filas: i recorre las filas.

3 • MATRIZ Y FORMA

La forma se deriva antes de ejecutar.

$$Y = XW + \mathbf{1}_B b^\top$$
$$Y : (B, H) = (3, 3)$$

SALIDA: 3 OBS. × 3 SALIDAS

4 • CÓDIGO Y CONTRASTE

PyTorch verifica una predicción previa.

$$Y = X @ W + b$$

```
assert Y.shape == (3, 3)
```

Contrastar forma, valores y significado.

5 • CIERRE DE LA CADENA

La misma relación debe sobrevivir en componentes, matrices y código.

CONTRASTE: FORMA + VALORES + SIGNIFICADO

INSUFICIENTE: «EL CÓDIGO CORRE»